

MEDIATEK

MT8676 Yocto DSI Panel User Manual

版本: 1.0
出版日期: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-08-12	涂红云	正式版

目录

版本记录	2
目录	3
1 概述	5
2 点屏前置条件	5
2.1 从屏厂或者串行器厂商获取如下信息	5
2.2 原理图审查	5
3 新增屏驱	6
3.1 Device 的修改	6
3.2 LK2 的修改	6
3.3 Kernel 的修改	7
4 重要事项	8
5 LK 驱动	8
5.1 增加 LK2 屏驱	9
5.2 编写驱动	9
6 Kernel DRM 驱动	15
6.1 增加 Kernel 驱动	16
6.2 编写驱动	16
7 精确的 FPS 设置	20
8 Debug SOP	21
8.1 Lk 阶段 Debug	21
8.2 Kernel 阶段 Debug	22
8.3 Resume 阶段 Debug	23
8.4 花屏 Debug	23
8.5 含有 Bridge 屏的 Debug	25
9 Dump Reg	25
9.1 LK 阶段	25
9.2 Kernel Dump reg	25
9.3 adb 命令 dump reg	26
9.4 Test Pattern 命令	26
9.5 分析 reg	27
10 使用 MTK max96789 panel driver	28

附件一 附加条款 37

1 DSI Panel

1.1 概述

本文主要介绍 Panel 驱动移植 SOP 以及相应的 debug 技巧。

1.2 点屏前置条件

1.2.1 从屏厂或者串行器厂商获取如下信息

- 屏厂要在治具上使用外部输入信号点亮屏
- 点屏前确定 “上下电时序、几个 port、几个 ddic、cmd/vdo mode、Cphy/dphy、Lane number”
- Init/deinit cmd（一定要是第一点提到的 cmd，即 “要在治具上使用外部输入信号点亮屏”）
- Width/height/vfp/vbp/vsa/hfp/hbp/hsa/fps/mipi clock 等等
- Bist mode cmd（可选，点不亮时要求提供）
- Dsc 参数，几个 slice（如果要开 dsc，请先根据参考初步判断一下 MTK IC 是否能支持）
- 其他功能如何实现（backlight/fps change/cabc...）
- 如果接串行器，串行器要如何设置等等

1.2.2 原理图审查

- 屏的电源如何控制
 - 如果使用 MTK regulator，请找 PMIC owner 询问如何控制屏的电源（LK、Kernel 配置方法都要获取到）
 - 用第三方 IC 请找第三方要
- GPIO 如何配置
 - 拿到 GPIO 功能表，DWS 先配置好
- 背光如何配置
 - Bring up 时可以直接控制 GPIO 拉高实现

1.3 新增屏驱

1.3.1 Device 的修改

1. 在 ko_order_table 中加入 ko 名称

```
device/mediateksample/auto8676p1_64_bsp/+/8628381/3/ko_order_table.csv
85 mtk_panel_ext.ko,../kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_panel_ext.ko
,ramdisk,Y,Y,user/userdebug/eng
86 bridge-serdes-max96789.ko,../kernel_device_modules-6.1/drivers/gpu/drm/panel/bridge-serdes-max96789
.ko,ramdisk,Y,Y,user/userdebug/eng
87 panel-serdes-max96789.ko,../kernel_device_modules-6.1/drivers/gpu/drm/panel/panel-serdes-max96789.k
o,ramdisk,Y,Y,user/userdebug/eng
```

2. 在 projectconfig.mk 中修改 bootlogo 设定

```
001: ARM_MTE_SUPPORT := .no
002: BOOT_LOGO := .vga
003: BUILD_LK := .yes
```

1.3.2 LK2 的修改

注意:

- 如果没有 LK，可以忽略此步骤

1. 添加新的驱动

```
M dev/lcm/include/lcm_drv.h
A dev/lcm/max96789_dsi_vdo/max96789_dsi_vdo.c
A dev/lcm/max96789_dsi_vdo/rules.mk
M dev/lcm/mt65xx_lcm_list.c
M platform/mediatek/mt6897/disp/disp_lcm.c
M project/auto8676p1_64_bsp_6p.mk
```

2. 在 \$(project).mk 中加入此驱动支持

```
project/auto8676p1_64_bsp_6p.mk
---
23 MODULE_DEPS += app/$(BOOT_APP)
24
25 MTK_IMPU_ENABLE := yes
26 MTK_LCM_COMMON_DRIVER_SUPPORT := no
27 MTK_LCM_LIST_SUPPORT="max96789_dsi_vdo"
28 MTK_FORCE_DISABLE_PRIM_DUAL_PIPE = yes
```

3. 在 lcm_list 中加入此驱动

```

/quark/lk2+/8627975/13/dev/lcm/mt65xx_lcm_list.c
34
35 LCM_DRIVER *lcm_driver_list[] = {
36
37 #if defined(MAX96789_DSI_VDO)
38     &max96789_dsi_vdo_lcm_drv,
39 #endif
40

```

4. 在 `lcmd_drv.h` 中加入此驱动

```

quark/lk2+/8627975/13/dev/lcm/include/lcm_drv.h
1082 extern LCM_DRIVER nt30072e_tmip_dsi_vdo_120hz_jd1
1083 extern LCM_DRIVER nt36532_wqxa2944_dsi_vdo_144hz
1084 extern LCM_DRIVER jd9365da_wxga_dsi_vdo_lcm_drv;
1085 extern LCM_DRIVER max96789_dsi_vdo_lcm_drv;
1086

```

1.3.2.1 Kernel 的修改

注意:

- Kernel-6.6 时修改类似，只是路径不一样

1. 添加新的驱动

← → ↻ gerrit.mediatek-inc.com/quark/kernel_device_modules-6.1/

A drivers/gpu/drm/panel/bridge-serdes-max96789.c

M drivers/gpu/drm/panel/Kconfig

M drivers/gpu/drm/panel/Makefile

A drivers/gpu/drm/panel/panel-serdes-max96789.c

2. 在 `kconfig` 加入新的驱动

```

/quark/kernel_device_modules-6.1+/8705673/8/drivers/gpu/drm/panel/Kconfig
24
25 config DRM_PANEL_SERDES
26 >     tristate "serdes max96789 vdo mode panel"
27 >     depends on OF
28 >     depends on DRM_MIPI_DSI
29 >     depends on BACKLIGHT_CLASS_DEVICE
30 >     help
31 >         Say Y here if you want to enable support for max96789 Serdes
32 >         vdo mode panel. The panel has a 1920x1080
33 >         resolution and uses 24-bits RGB per pixel. It provides a
34 >         MIPI DSI interface to host.

```

3. 在 `makefile` 中加入新的驱动

```

/kernel_device_modules-6.1+/8705673/8/drivers/gpu/drm/panel/Makefile
:9 obj-$(CONFIG_DRM_PANEL_ILI7838E_DV2_SPR_CMD) += k6989v1_64_alpha/panel-ili7838e-dv2
:0 endif
:1
:2 obj-$(CONFIG_DRM_PANEL_SERDES) += panel-serdes-max96789.o bridge-serdes-max96789.o

```

4. 在 `kernel config` 中加入驱动

```

kernel_device_modules-6.1+/8705663/2/arch/arm64/configs/mgk_64_k61_defconfig
3 CONFIG_DRM_PANEL_BOE_JD9365DA=m
4 CONFIG_DRM_PANEL_SERDES=m
5 CONFIG_DRM_PANEL_TRULY_TD4330_VDO=m

```

5. 在 kleaf 中加入驱动

```

/kernel_device_modules-6.1+/8705631/2/kernel/kleaf/mgk_64_k61.bzl
126 "drivers/gpu/drm/mediatek/mtk-mml.ko",
127 "drivers/gpu/drm/mediatek/mtk-mml.ko",
128 "drivers/gpu/drm/panel/bridge-serdes-max96789.ko",
129 "drivers/gpu/drm/panel/k6985v1_64_alpha/panel-nt37705-al
58 "drivers/gpu/drm/panel/panel-samsung-s68fc01-vdo-aod.ko",
59 "drivers/gpu/drm/panel/panel-serdes-max96789.ko",
60 "drivers/gpu/drm/panel/panel-sc-nt36672c-vdo-120hz.ko",

```

6. 在 dts 中加入驱动

```

12/arch/arm64/boot/dts/mediatek/mt8676p1_64_bsp_6p.dts
=
status = "okay";
};
&i2c1 {
status = "okay";
max96789: max96789@48 {
compatible = "maxim,max96789";
reg = <0x48>;
reset-gpios = <&pio 37 0>;
pinctrl-names = "default";
setting = <&setting_compatible>;
init-in-lk = <1>;
};
};
7 &dsi0 {
8 status = "okay";
9 #address-cells = <1>;
0 #size-cells = <0>;
1 panel1@0 {
2 compatible = "lcm,dsi,max96789";
3 reg = <0>;
4
5 ser = <&max96789>;
6
7 port {
8 panel_in1: endpoint {
9 remote-endpoint = <&dsi_out>;
0
1 };
};
};

```

1.4 重要事项

- 为了防止从 lk 转 Kernel 画面抖动，kernel 启动后 dsi/mipi_tx/panel 的设置都是用 lk 的参数，suspend/resume 之后才是全部采用 Kernel 的配置。如果发现 suspend/resume 之后有参数跟上电启动不同，请检查这三部分的设置区别
- Kernel 启动时会获取一些参数给 display 使用，在 Kernel panel driver 没有 ready 的情况下系统不能正常工作是正常现象
- 后面页面中写的 must set 的参数是必须要写的，其他部分根据需要填写

1.5 LK 驱动

LK2 路径: /vendor/mediatek/proprietary/bootable/bootloader/lk2/

Name	Path
Display Core	platform/mediatek/mt6xxx/disp
Panel driver	dev/lcm/
LOGO picture	LK2: Vendor/mediatek/proprietary/external/BootLogo/logo LK: vendor/mediatek/proprietary/bootable/bootloader/lk/dev/logo

1.5.1 增加 LK2 屏驱

参考第 1.3.2 章新增 LK 屏驱文件和修改。

1.5.2 编写驱动

1. 编写 lcm_drv 结构体 (must set)

- 以<tv101wum_n16_wuxga_dsi_video_boe_rtq6752.c> 为例:

```
LCM_DRIVER tv101wum_n16_wuxga_dsi_vdo_boe_rtq6752_drv = {
    .name = "tv101wum_n16_wuxga_dsi_vdo_boe_rtq6752_drv",
    .set_util_funcs = lcm_set_util_funcs,
    .get_params = lcm_get_params,
    .init = lcm_init,
    .suspend = lcm_suspend,
    .resume = lcm_resume,
    .init_power = lcm_init_power,
    .compare_id = lcm_compare_id,
    .resume_power = lcm_resume_power,
    .suspend_power = lcm_suspend_power,
    .ata_check = lcm_ata_check,
    .update = lcm_update,
};
```

2. 填充 LCM 参数 (must set)

- 按照实际参数参考其他屏驱填写

▪ Lcm 宽和高 (must set)

```
static void lcm_get_params(LCM_PARAMS *params)
{
    memset(params, 0, sizeof(LCM_PARAMS));

    params->type = LCM_TYPE_DSI;
    params->width = FRAME_WIDTH;
    params->height = FRAME_HEIGHT;
```

▪ Lcm 参数 (dsi 设置, must set)

```
params->dsi.mode = SYNC_PULSE_VDO_MODE;
params->dsi.switch_mode = CMD_MODE;
params->dsi.switch_mode_enable = 0;

/* DSI */
/* Command mode setting */
params->dsi.LANE_NUM = LCM_FOUR_LANE;
/* The following defined the fomat for data coming from LCD engine.
params->dsi.data_format.color_order = LCM_COLOR_ORDER_RGB;
params->dsi.data_format.trans_seq = LCM_DSI_TRANS_SEQ_MSB_FIRST;
params->dsi.data_format.padding = LCM_DSI_PADDING_ON_LSB;
params->dsi.data_format.format = LCM_DSI_FORMAT_RGB888;

/* Highly depends on LCD driver capability. */
params->dsi.packet_size = 256;
```

▪ Lcm 参数 (vdo timing, must set)

```

params->dsi.vertical_sync_active = VSA;
params->dsi.vertical_backporch = VBP;
params->dsi.vertical_frontporch = VFP;
params->dsi.vertical_frontporch_for_low_power = 750;
params->dsi.vertical_active_line = FRAME_HEIGHT;

params->dsi.horizontal_sync_active = HSA;
params->dsi.horizontal_backporch = HBP;
params->dsi.horizontal_frontporch = HFP;
params->dsi.horizontal_active_pixel = FRAME_WIDTH;
params->dsi.ssc_disable = 1;

```

- esd 配置 (optional, 使用 bridge 必须关掉)

```

params->dsi.esd_check_enable = 1;
params->dsi.customization_esd_check_enable = 0;
params->dsi.lcm_esd_check_table[0].cmd = 0x0A;
params->dsi.lcm_esd_check_table[0].count = 1;
params->dsi.lcm_esd_check_table[0].para_list[0] = 0x9C;

```

- pll config (PLL_CLOCK 和 data_rate 至少要设置 1 个, 参考下面公式计算)

(DPHY)
$$\text{Data Rate} = \frac{\text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times \text{Blanking Ratio} \times \text{Bit Per Pixel}}{\text{Lane Num}}$$

(DPHY DSC)
$$\text{Data Rate} = \frac{\text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times \text{Compression Ratio} \times \text{Bit Per Pixel} + \text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times (\text{Blanking Ratio} - 1) \times \text{Bit Per Pixel}}{\text{Lane Num}}$$

(CPHY)
$$\text{Symbol Rate} = \frac{\text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times \text{Blanking Ratio} \times \text{Bit Per Pixel} \times 7/16}{\text{Trio Num}}$$

(CPHY DSC)
$$\text{Symbol Rate} = \frac{\text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times \text{Compression Ratio} \times \text{Bit Per Pixel} \times 7/16 + \text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times (\text{Blanking Ratio} - 1) \times \text{Bit Per Pixel} \times 7/16}{\text{Trio Num}}$$

```
00184: ...params->dsi.data_rate = 563;
```

- 展频配置 (optional, 使用 bridge 必须关掉)

MIPI Clock Spread Spectrum:

```

params->dsi.ssc_disable = 0;
params->dsi.ssc_range = 4;

```

Ssc_disable: 0 → enable ssc, 1 → disable ssc
Ssc_range: 1~8, default value:0

- mipi clock 配置 (optional)

Mipi clk mode setting:

- non-continous clk mode:

```

params->dsi.cont_clock = 0;
params->dsi.clk_lp_per_line_enable = 1;

```

- continuous clk mode:

```

params->dsi.cont_clock = 1;
/* params->dsi.clk_lp_per_line_enable = 1; */

```

- cphy/dphy 配置 (by case, cphy 设置为 1, dphy 设置为 0)

```
params->dsi.isCphy = 1;
```

- dual port 设置 (by case)

```

params->lcm_if = LCM_INTERFACE_DSI0;
params->lcm_cmd_if = LCM_INTERFACE_DSI_DUAL;

params->dsi.mode = CMD_MODE;

params->dsi.output_mode = MTK_PANEL_DUAL_PORT;
params->dsi.ssc_disable = 1;

```

- Dsc 配置 (by case)

- 如果要用 DSC, 必须设置 dsc_enable=1, 并设定 DSC 参数
- 是否需要支持 DSC 取决于屏, 且 DSC 参数需要屏厂提供

- MTK 特有参数 dsc_cfg:
 - a) 8bpc to 8bpp 设置为 0x22
 - b) 10bpc to 8bpp 设置为 0x828
- 其他参数——对应到屏厂提供参数
- 注意观察 rc_buf_thresh 值的范围是 14-126, 如果厂商提供的远大于这个, 将厂商数据右移 6 位
- dsc 设定例子

```

params->dsi.dsc_enable = 1;
params->dsi.dsc_params.ver = 17;
params->dsi.dsc_params.slice_mode = 1;
params->dsi.dsc_params.rgb_swap = 0;
params->dsi.dsc_params.dsc_cfg = 34;
params->dsi.dsc_params.rct_on = 1;
params->dsi.dsc_params.bit_per_channel = 8;
params->dsi.dsc_params.dsc_line_buf_depth = 9;
params->dsi.dsc_params.bp_enable = 1;
params->dsi.dsc_params.bit_per_pixel = 128;
params->dsi.dsc_params.pic_height = 2400;
params->dsi.dsc_params.pic_width = 1080;
params->dsi.dsc_params.slice_height = 8;
params->dsi.dsc_params.slice_width = 540;
params->dsi.dsc_params.chunk_size = 540;
params->dsi.dsc_params.xmit_delay = 170;
params->dsi.dsc_params.dec_delay = 526;
params->dsi.dsc_params.scale_value = 32;
params->dsi.dsc_params.increment_interval = 43;
params->dsi.dsc_params.decrement_interval = 7;
params->dsi.dsc_params.line_bpg_offset = 12;
params->dsi.dsc_params.nfl_bpg_offset = 3511;
params->dsi.dsc_params.slice_bpg_offset = 3255;
params->dsi.dsc_params.initial_offset = 6144;
params->dsi.dsc_params.final_offset = 7072;
params->dsi.dsc_params.flatness_minqp = 3;
params->dsi.dsc_params.flatness_maxqp = 12;
params->dsi.dsc_params.rc_model_size = 8192;
params->dsi.dsc_params.rc_edge_factor = 6;
params->dsi.dsc_params.rc_quant_incr_limit0 = 11;
params->dsi.dsc_params.rc_quant_incr_limit1 = 11;
params->dsi.dsc_params.rc_tgt_offset_hi = 3;
params->dsi.dsc_params.rc_tgt_offset_lo = 3;

```

- 厂商提供的 pps 参数例子 1

800 2560 slice 20		Dec	Hex
force reg_dsc_version_major[3:0]	=	1	1
force reg_dsc_version_minor[3:0]	=	1	1
force reg_pps_identifier[7:0]	=	0	00
force reg_bits_per_component[3:0]	=	8	8
force reg_linebuf_depth[3:0]	=	9	9
force reg_block_pred_enable	=	1	1
force reg_convert_rgb	=	1	1
force reg_simple_422	=	0	0
force reg_vbr_enable	=	0	0
force reg_bits_per_pixel[9:0]	=	128	80
force reg_pic_height[15:0]	=	2560	0A00
force reg_pic_width[15:0]	=	800	0320
force reg_slice_height[15:0]	=	20	14
force reg_slice_width[15:0]	=	800	0320
force reg_chunk_size[15:0]	=	800	0320
force reg_initial_xmit_delay[9:0]	=	512	0200
force reg_initial_dec_delay[15:0]	=	657	0291
force reg_initial_scale_value[5:0]	=	32	0020

- 厂商提供的 pps 参数例子 2

```

unsigned char pps[] = {0x11, 0x00, 0x00, 0xA8, 0x30, 0x80, 0x07, 0x54,
0x08, 0xB8, 0x00, 0x14, 0x02, 0xEE, 0x02, 0xEE,
0x02, 0x00, 0x01, 0x8C, 0x00, 0x20, 0x00, 0x90,
0x00, 0x0A, 0x00, 0x06, 0x10, 0x00, 0x12, 0x4E,
0x18, 0x00, 0x10, 0xF0, 0x07, 0x10, 0x20, 0x00,
0x06, 0x0F, 0x0F, 0x33, 0x0E, 0x1C, 0x2A, 0x36,
0x46, 0x54, 0x62, 0x69, 0x70, 0x77, 0x79, 0x7B,
0x7D, 0x7E, 0x02, 0x02, 0x22, 0x00, 0x2A, 0x40,
0x2A, 0xBE, 0x3A, 0xFC, 0x3A, 0xFA, 0x3A, 0xF8,
0x3B, 0x3B, 0x3B, 0x78, 0x3B, 0xB6, 0x4B, 0xB6,
0x4B, 0xF4, 0x4B, 0xF4, 0x6C, 0x34, 0x84, 0x74,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};

```

GPIO 配置 (参考 GPIO SOP)

ADC	ClockBuffer	EINT	GPIO	I2C	KEYPAD	MD1_EINT	PMIC		
ID	EintMode	Def.Mode	InPull En	InPull SelfHigh	Def.Dir	OutHigh	VarName1	VarName2	
39	GPIO39	☐	4:MD_UTXD1	☐					
40	GPIO40	☐	1:DISP_PWM	☐			GPIO_PWM_BL_PIN		
41	GPIO41	☐	1:DSI_TF	☒			GPIO_DSI_TF_PIN		
42	GPIO42	☐	0:GPIO42			OUT	GPIO_LCM_RST		
43	GPIO43	☐	1:MD_INT1_...	☒	☒		GPIO_SIM1_HOT_PLUG		
44	GPIO44	☐	NC	☒		IN			
45	GPIO45	☐	1:MD1_SIM...	☐			GPIO_SIM2_SCLK		
46	GPIO46	☐	1:MD1_SIM...	☐			GPIO_SIM2_SRST		

```

static void lcm_set_reset_pin(UINT32 value)
{
    int rst_pin;

    rst_pin = lcm_util_get_pin("lcm_rst_out0_gpio");
    if (rst_pin < 0) {
        OUTREG32(MMSYS_CONFIG_BASE+0x150, value);
        dprintf(0, "reg set lcm rst pin\n");
    } else {
        dprintf(0, "gpio set lcm rst pin\n");
        rst_pin |= PIN_ENCRYPT;
        mt_set_gpio_mode(rst_pin, GPIO_MODE_00);
        mt_set_gpio_dir(rst_pin, GPIO_DIR_OUT);
        if(value)
            mt_set_gpio_out(rst_pin, GPIO_OUT_ONE);
        else
            mt_set_gpio_out(rst_pin, GPIO_OUT_ZERO);
    }
}

static void lcm_set_reset_pin(UINT32 value)
{
    smtkfb {
        pinctrl-names = "lcd_bias_enp1_gpio", "lcd_bias_enp0_gpio",
        "lcd_bias_enp1_gpio", "lcd_bias_enp0_gpio",
        "lcm_rst_out1_gpio", "lcm_rst_out0_gpio",
        "mode_te_te";
        pinctrl-0 = <smtkfb_pins_lcd_bias_enp1>;
        pinctrl-1 = <smtkfb_pins_lcd_bias_enp0>;
        pinctrl-2 = <smtkfb_pins_lcd_bias_enp1>;
        pinctrl-3 = <smtkfb_pins_lcd_bias_enp0>;
        pinctrl-4 = <smtkfb_pins_lcd_bias_enp1>;
        pinctrl-5 = <smtkfb_pins_lcd_bias_enp0>;
        pinctrl-6 = <smtkfb_pins_lcd_bias_enp1>;
        status = "okay";

        mtkfb_pins_lcd_rst_out0_gpio: lcm_rst_out0_gpio {
            pins_cmd_dat {
                pinmux = <PINMUX_GPIO42_FUNC_GPIO42>;
                slew-rate = <1>;
                output-low;
            };
        };
    };
}

```

3. 编写 LCM init power 函数 (must set)

```

static void lcm_init_power(void)
{
    SET_RESET_PIN(0);
    MDELAY(100);
    if (lcm_util.set_gpio_lcd_enp_bias) {
        lcm_util.set_gpio_lcd_enp_bias(1);
        i2c_w8(0x06, 0, 0x3e, 0x00, 0x0f);
        i2c_w8(0x06, 0, 0x3e, 0x01, 0x0f);

        LCM_LOGI("[zsq] ----lcm_init_power-----\n");
    }
    else
        LCM_LOGI("[zsq] set_gpio_lcd_enp_bias not defined\n");
}

```

- 按照 hw 设计实际情况编写

4. 编写 LCM init 函数 (must set)

```

4:
5: static void lcm_init(void *fdt)
6: {
7:     dprintf(0, "%s +++\n", __func__);
8:     SET_RESET_PIN(1, fdt);
9:     MDELAY(5);
10:    SET_RESET_PIN(0, fdt);
11:    MDELAY(15);
12:    SET_RESET_PIN(1, fdt);
13:    MDELAY(60);
14:
15:    is_send_data = false;
16:    push_table_by_type(lcm_bkl_setting,
17:        sizeof(lcm_bkl_setting) / sizeof(struct cust_setting_table),
18:        1);
19:    #if LCM_USED_2PORT_4SLICE
20:    push_table_by_type(lcm_pps_setting,
21:        sizeof(lcm_pps_setting) / sizeof(struct cust_setting_table),
22:        1);
23:    MDELAY(20);
24:    push_table_by_type(lcm_4slice_setting,
25:        sizeof(lcm_4slice_setting) / sizeof(struct cust_setting_table),
26:        1);
27:    MDELAY(20);
28:    #endif
29:    push_table(lcm_initialization_setting,
30:        sizeof(lcm_initialization_setting) / sizeof(struct LCM_setting_table),
31:        1);
32:
33:    dprintf(0, "%s ---\n", __func__);
34: } ? end lcm_init ?
35:

```

- 按照屏的时序编写

5. 编写 LCM suspend/resume 函数 (optional)

```

static void lcm_suspend(void)
{
    int ret;

    push_table(lcm_suspend_setting,
        sizeof(lcm_suspend_setting) / sizeof(struct LCM_setting_table), 1);

    SET_RESET_PIN(0);
    MDELAY(10);
}

static void lcm_resume(void)
{
    lcm_init();
}

```

6. 编写 LCM compare 函数 (如果是多屏兼容才需要)

```

#define LCM_ID_NT36672C_JDI (0x41)

static unsigned int lcm_compare_id(void)
{
    unsigned int id = 0;
    unsigned char buffer[2];
    unsigned int array[16];

    LCM_LOGI("%S: enter\n", __func__);

    SET_RESET_PIN(1);
    MDELAY(10);
    SET_RESET_PIN(0);
    MDELAY(10);
    SET_RESET_PIN(1);
    MDELAY(10);

    array[0] = 0x00013700; /* read id return two byte, version and id */
    dsi_set_cmdq(array, 1, 1);

    read_reg_v2(0xDA, buffer, 1);
    id = buffer[0]; /* we only need ID */

    LCM_LOGI("%s, nt36672c_id_jdi=0x%08x\n", __func__, id);

    if (id == LCM_ID_NT36672C_JDI)
        return 1;
    else
        return 0;
} ? end lcm_compare_id ?

```

7. 编写背光函数 (使用 lcm pwm 才需要)

```
static void lcm_setbacklight(unsigned int level)
{
    LCM_LOGI("%s,nt35695 backlight: level = %d\n", __func__, level);
    bl_level[0].para_list[1] = level;
    push_table(bl_level, sizeof(bl_level) / sizeof(struct LCM_setting_table), 1);
}
```

```
static struct LCM_setting_table lcm_suspend_setting[] = {
    {0x28, 0, {} },
    {0x10, 0, {} },
    {REGFLAG_DELAY, 120, {} },
    {0x4F, 1, {0x01} },
    {REGFLAG_DELAY, 120, {} }
};

static struct LCM_setting_table init_setting[] = {
    {0xFF, 1, {0x24} },
    {0xFB, 1, {0x01} },
    {0x2D, 1, {0x08} },
    {0xFF, 1, {0x24} }, /* Return To CMD1 */
    {0x6E, 1, {0x10} }, /* Return To CMD1 */
    {0xFB, 1, {0x01} }, /* Return To CMD1 */
    {0xFF, 1, {0x10} }, /* Return To CMD1 */

    {0xFF, 1, {0x10} }, /* Return To CMD1 */
    {REGFLAG_UDELAY, 1, {} },
#ifdef LCM_DSI_CMD_MODE
    {0xBB, 1, {0x10} }, /*CMD MODE*/
#else
    {0xBB, 1, {0x03} }, /*VDO MODE*/
#endif
    {0x3B, 5, {0x03, 0x0A, 0x0A, 0x0A, 0x0A} },
    {0x53, 1, {0x24} },
    {0x55, 1, {0x00} },
    {0x5E, 1, {0x00} },
    {0xFF, 1, {0x24} }, /* CMD2 Page4 Entrance */
    {REGFLAG_UDELAY, 1, {} },
    {0xFB, 1, {0x01} },
}
```

8. 填充初始化参数 (must set)

9. 参考其他驱动编写其余重要部分 (must set)

```

#define GPIO_LED_BIAS_EN GPIO87
#define GPIO_LCD_RESET GPIO85
#define GPIO_LCM_ID0 GPIO109
#define GPIO_LCM_ID1 GPIO98
#define GPIO_LCD_BIAS_ENP GPIO82
#define GPIO_LCD_BIAS_ENN GPIO81
static const unsigned int BL_MIN_LEVEL = 20;
static LCM_UTIL_FUNCS lcm_util;
#define BIAS_I2C_ADDRESS (0x11)

#define SET_RESET_PIN(v, fdt) (lcm_util.set_reset_pin(v, fdt))
#define MDELAY(n) (lcm_util.mdelay(n))
#define UDELAY(n) (lcm_util.udelay(n))

/* ----- */
/* Local Functions */
/* ----- */

#define dsi_set_cmdq_V2(cmd, count, ppara, force_update) \
    lcm_util.dsi_set_cmdq_V2(cmd, count, ppara, force_update)
#define dsi_set_cmdq(pdata, queue_size, force_update) \
    lcm_util.dsi_set_cmdq(pdata, queue_size, force_update)
#define write_cmd(cmd) lcm_util.dsi_write_cmd(cmd)
#define write_regs(addr, pdata, byte_nums) \
    lcm_util.dsi_write_regs(addr, pdata, byte_nums)
#define read_reg(cmd) \
    lcm_util.dsi_dcs_read_lcm_reg(cmd)
#define read_reg_v2(cmd, buffer, buffer_size) \
    lcm_util.dsi_dcs_read_lcm_reg_v2(cmd, buffer, buffer_size)

/* ----- */
/* Local Constants */
/* ----- */
#define LCM_DSI_CMD_MODE 0
#define FRAME_WIDTH (1200)
#define FRAME_HEIGHT (2000)

#define REGFLAG_DELAY 0xFFFC
#define REGFLAG_UELAY 0xFFFB
#define REGFLAG_END_OF_TABLE 0xFFFF
#define REGFLAG_RESET_LOW 0xFFFE
#define REGFLAG_RESET_HIGH 0xFFFF

static void lcm_set_gpio_output(unsigned int GPIO, unsigned int output)
{
    mt_set_gpio_mode(GPIO, GPIO_MODE_00);
    mt_set_gpio_dir(GPIO, GPIO_DIR_OUT);
    mt_set_gpio_out(GPIO, (output > 0) ? GPIO_OUT_ONE : GPIO_OUT_ZERO);
}

struct LCM_setting_table {
    unsigned int cmd;
    unsigned char count;
    unsigned char para_list[200];
};

static void push_table(struct LCM_setting_table *table, unsigned int count,
    unsigned char force_update)
{
    unsigned int i;
    unsigned int cmd;
    for (i = 0; i < count; i++) {
        cmd = table[i].cmd;
        switch (cmd) {
            case REGFLAG_DELAY:
                if (table[i].count <= 10)
                    MDELAY(table[i].count);
                else
                    MDELAY(table[i].count);
                break;
            case REGFLAG_UELAY:
                UDELAY(table[i].count);
                break;
            case REGFLAG_END_OF_TABLE:
                break;
            default:
                dsi_set_cmdq_V2(cmd, table[i].count, table[i].para_list,
                    force_update);
                break;
        }
    }
} > end for i=0;i<count;i++ ?
} > end push_table ?

static void lcm_set_util_funcs(const LCM_UTIL_FUNCS *util)
{
    memcpy(&lcm_util, util, sizeof(LCM_UTIL_FUNCS));
}

```

1.6 Kernel DRM 驱动

文件路径: /kernel/ kernel_device_modules-6.1/drivers/gpu/drm/mediatek/Mediatek_v2

Name	Files
DRM framework relative	mtk_drm_drv.c, mtk_drm_crtc.c, mtk_drm_fb.c, mtk_drm_gem.c, mtk_drm_plane.c...
MTK DDP	mtk_disp_ccorr.c , mtk_disp_gamma.c, mtk_disp_dither.c, mtk_disp_ovl.c, mtk_disp_rdma.c, mtk_disp_dsi.c , mtk_disp_wdma.c ...
Connector	mtk_dsi.c, mtk_dpi.c, mtk_hdmi.c, mtk_mipi_tx.c...

Name	Path
DRM Core	drivers/gpu/drm/
MTK implement	drivers/gpu/drm/mediatek/mediatek_v2 (K510) OR drivers/gpu/drm/mediatek/ (K419)
Panel driver	drivers/gpu/drm/panel/
DRM Header File	include/drm/ include/uapi/drm/

1.6.1 增加 Kernel 驱动

参考章节 1.3.2.1 新增修改 Kernel 驱动。

1.6.2 编写驱动

1. 填充 Kernel 驱动结构体 (must set)

- DRM 原始结构体

```
static const struct drm_display_mode default_mode = {
    .clock = 422163,
    .hdisplay = 1080,
    .hsync_start = 1080 + 274,
    .hsync_end = 1080 + 274 + 12,
    .htotal = 1080 + 274 + 12 + 56,
    .vdisplay = 2400,
    .vsync_start = 2400 + 2528,
    .vsync_end = 2400 + 2528 + 10,
    .vtotal = 2400 + 2528 + 10 + 10,
    .vrefresh = 60,
};
```

2. 编写 MTK 扩展结构体 (must set)

```
static struct mtk_panel_params ext_params = {
    .pll_clk = 369,
    .data_rate = 738,
    .vfp_low_power = 4178,
    .cust_esd_check = 0,
    .esd_check_enable = 1,
    .lcm_esd_check_table[0] = {
        .cmd = 0x0A, .count = 1, .para_list[0] = 0x9C,
    },
    .is_cphy = 1,
};
```

- 在 super frame 模式下, 必须设置 the pll_clk/crop_width/crop_height/physical_width/physical_height, 否则必须设置 pll_clk
- Pll_clk: mipi 时钟

- Crop width/crop height: 在 super frame 模式下, Crop width/crop height 应该被设置为对应于其中一个 panel 的宽度/高度
- Physical_width/physical_height: 在 super frame 模式下, physical_width 应该被设置为两个 panel 的宽度之和。physical_height 应该被设置为两个 panel 中较大的那个高度。

3. 填充 DRM 架构驱动函数 (must set)

- DRM 原始架构函数

```
static const struct drm_panel_funcs jdi_drm_funcs = {
    .disable = jdi_disable,
    .unprepare = jdi_unprepare,
    .prepare = jdi_prepare,
    .enable = jdi_enable,
    .get_modes = jdi_get_modes,
};
```

4. 编写 probe 函数 (must set)

```
static int lcm_probe(struct mipi_dsi_device *dsi)
{
    struct device *dev = &dsi->dev;
    struct device_node *dsi_node, *remote_node = NULL, *endpoint = NULL, *ser_node;
    const phandle *ser_phandle;
    struct lcm *ctx;
    int ret;
    const struct panel_desc *desc;

    pr_info("%s+, name:%s\n", __func__, dev->of_node->name);

    dsi_node = of_get_parent(dev->of_node);
    if (dsi_node) {
        endpoint = of_graph_get_next_endpoint(dsi_node, NULL);
        if (endpoint) {
            remote_node = of_graph_get_remote_port_parent(endpoint);
            if (!remote_node) {
                pr_info("No panel connected, skip probe lcm\n");
                return -ENODEV;
            }
            pr_info("device node name:%s\n", remote_node->name);
        }
    }
    if (remote_node != dev->of_node) {
        pr_info("%s+ skip probe due to not current lcm\n", __func__);
        return -ENODEV;
    }
}
```

5. 编写 remove 函数 (must set)

```

}; static void lcm_remove(struct mipi_dsi_device *dsi)
{
    struct lcm *ctx = mipi_dsi_get_drvdata(dsi);
    #if defined(CONFIG_MTK_PANEL_EXT)
    struct mtk_panel_ctx *ext_ctx = find_panel_ctx(&ctx->panel);
    #endif

    mipi_dsi_detach(dsi);
    drm_panel_remove(&ctx->panel);
    #if defined(CONFIG_MTK_PANEL_EXT)
    mtk_panel_detach(ext_ctx);
    mtk_panel_remove(ext_ctx);
    #endif
}

```

6. 编写 prepare 函数 (must set)

```

static int lcm_prepare(struct drm_panel *panel)
{
    struct lcm *ctx = panel_to_lcm(panel);

    pr_info("%s +\n", __func__);

    lcm_poweron(ctx);
    lcm_panel_init(ctx);
    serdes_pre_enable(ctx->bridge);

    #if defined(CONFIG_MTK_PANEL_EXT)
    mtk_panel_tch_rst(panel);
    #endif
    pr_info("%s -\n", __func__);

    return 0;
}

```

7. 编写 init 函数 (must set)

```

static void lcm_panel_init(struct lcm *ctx)
{
    pr_info("%s +\n", __func__);
    pr_info("%s -\n", __func__);
}

```

8. 编写 enable 函数 (must set)

```

static int lcm_enable(struct drm_panel *panel)
{
    struct lcm *ctx = panel_to_lcm(panel);

    pr_info("%s +\n", __func__);

    serdes_enable(ctx->bridge);

    pr_info("%s -\n", __func__);

    return 0;
}

```

9. 编写 disable 函数 (must set)

```
static int lcm_disable(struct drm_panel *panel)
{
    struct lcm *ctx = panel_to_lcm(panel);
    pr_info("%s +\n", __func__);
    serdes_disable(ctx->bridge);
    pr_info("%s -\n", __func__);
    return 0;
}
```

10. 编写 unprepare 函数 (must set)

```
static int lcm_unprepare(struct drm_panel *panel)
{
    struct lcm *ctx = panel_to_lcm(panel);
    pr_info("%s +\n", __func__);
    lcm_poweroff(ctx);
    pr_info("%s -\n", __func__);
    return 0;
}
```

11. 编写 get_modes 函数 (must set)

```
static int lcm_get_modes(struct drm_panel *panel,
    struct drm_connector *connector)
{
    struct drm_display_mode *mode;

    pr_info("%s +\n", __func__);
    mode = drm_mode_duplicate(connector->dev, &default_mode);
    if (!mode) {
        pr_info("failed to add mode %ux%ux@%u\n",
            default_mode.hdisplay,
            default_mode.vdisplay,
            drm_mode_vrefresh(&default_mode));
        return -ENOMEM;
    }

    mode->type = DRM_MODE_TYPE_DRIVER | DRM_MODE_TYPE_PREFERRED;
    drm_mode_set_name(mode);
    drm_mode_probed_add(connector, mode);

    connector->display_info.width_mm = default_mode.width_mm;
    connector->display_info.height_mm = default_mode.height_mm;

    pr_info("%s -\n", __func__);

    return 1;
} « end lcm_get_modes »
```

12. 编写 MTK 扩展函数 reset (optional)

13. 编写 MTK 扩展函数 set_backlight_cmdq function (by case)

14. 编写 MTK 扩展函数 ext_param_set function (by case)

15. 编写 MTK 扩展函数 ata_check function (optional)

1.7 精确的 FPS 设置

- 手写笔应用中，需要 FPS 非常精准（120+/-0.01），通过 data_rate_khz 来实现
- 首先通过表格（找 PM 要）计算出近似值（ideal bit freq）填到 data_rate，再通过 data_rate_khz 进行微调
- 注意即使设置了 data_rate_khz，data_rate/pll_clk 也需要设置（不管是 LK 还是 kernel 都是）
- Data_rate_khz 需要微调后实测 FPS，目前不能通过理论计算
- Vdo_per_frame_lp_enable 对应下面表格 video mod e keep hs mode，此项对 vdo mode fps 影响比较大

输入参数	不可动	可自行填入						
1 DSI Frame Size		Vertical (unit : line)				Horizontal (unit : pixel) 只有HFP可调整求frame rate，其他都按照客户需		
H_Total (pixel)	V_Total (line)	VACT	VFP	VPW	VBP	HACT	HFP	HSA
719	1962	1840	26	2	94	1472	110	10
Option							Real Horizontal (unit: pixel)	
DSI_BLLP_WC	Video Mode Keep HS Mode	Command Mode Keep HS Mode	Null Packet Word Count (Command Mode)	PROGSEQ_SKIP_EN	PHY Timing Post Processing	Display Pipe	HFP	HSA
10	Y	HS mode w/ Null Packet	13	0	1	1	113.33	10.00

需求	O: D-PHY 1: C-PHY	D-PHY: 1~4 C-PHY: 1~3	Compression Ratio	24: RGB888 30: RGB101010	range : 0~3	Bit Rate (D-PHY) Symbol Rate (C-PHY)
HBP	C/D-PHY	lane_num	DSC	PixelBit (bit/pixel)	mode selector	Ideal Bit_Freq
108	0	4	0.3333	24	1	1226
el)	PHY Timing Spec. (unit: UI)		Unit: Frame per Second(FPS)	Line Time Unit: us	Frame Time Unit: ms	Unit: MHz
HBP	t3-prebegin	t3-post	Real Frame Rate (FPS)	Real LineTime	Real FrameTime	Disp Clock
108.67	0	0	144.1127	3.537	6.939	457.8273063

```
00397: 1;  
00398:  
00399: struct mtk_panel_params {  
00400:     unsigned int change_fps_by_vfp_send_cmd;  
00401:     unsigned int pll_clk;  
00402:     unsigned int data_rate;  
00403:     /* t3-prebegin, t3-post must be set, event if data_rate_khz is set  
00404:     unsigned int data_rate_khz; //only used in exact value for fps  
00405:     unsigned int vdo_per_frame_lp_enable; /* Enable video mode per frame lp */  
00406:  
00855:     .data_rate_khz = 960298,  
00856:     .data_rate = 960,  
00857: }
```

1.8 Debug SOP

1.8.1 Lk 阶段 Debug

- LK 屏不亮
 - 背光是否亮?
 - 是, 继续下面步骤
 - 否, 检测背光控制方式是否正确 (LK 也会采用 dts 里面的设置)
 - 也可采用直接拉高 GPIO 的方式先避开
 - 如果使用背光 IC, 询问对应 vendor 要如何控制 disp_pwm mode
 - 背光控制模式是否正确

a) Disp_pwm 模式

```
pwmleds: pwmleds {
    compatible = "mediatek,pwm-leds";
    status = "disabled";
    backlight {
        label = "lcd-backlight";
        pwms = <&disp_pwm0 0 500000>;
        max-brightness = <255>;
        pwm-names = "lcd-backlight";
    };
};
```

b) I2C 模式

```
mtk_leds: mtk_leds {
    backlight {
        label = "lcd-backlight";
        max-brightness = <255>;
        max-hw-brightness = <2047>;
    };
};

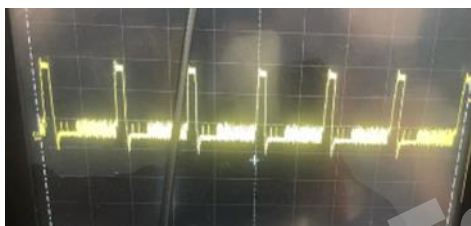
&mtk_leds {
    compatible = "mediatek,i2c-leds";
    backlight {
        led_mode = <6>;
        gate_enable = <1>;
        pwm_config = <0 1 0 0 0>;
    };
};
```

c) Lcm pwm mode

```
1:
2: &mtk_leds {
3:     compatible = "mediatek,disp-leds";
4:
5:     backlight {
6:         led_mode = <4>;
7:         gate_enable = <1>;
8:         pwm_config = <0 1 0 0 0>;
9:     };
0: };
```

- 屏所需要的电压是否有正常供上
 - 是, 继续下面步骤
 - 否, 用示波器量各路电压, 看哪一路不正常, 询问 PMIC owner 要如何控制
 - 上电时序是否符合屏的要求
 - 用示波器量信号

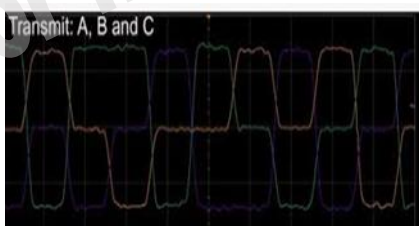
- Init cmd 是否有发成功？（判断标志，读屏的 0x0A 为非 0 值，需屏支持）
 - 是，继续下面步骤
 - 否
 - 发送 cmd 是用 generic？还是 dcs？还是混用？默认 cmd<0xb0 用 dcs，cmd>=0xb0 用 generic
 - 找屏厂要屏的 BIST pattern 命令，看 BIST 是否能正常出来
 - Bist 能亮说明屏的供电没问题
 - Bist 也不亮
 - 是否有 lane swap？量 D0P/N 是否有波形出来
 - init cmd 是否给错，用逻辑分析仪抓发送 init cmd 的数据给屏厂检查
 - 如果使用 bridge，检测 bridge 的状态寄存器提供信息给 bridge 厂商分析
- Mipi 信号是否正常（做 mipi cts 测试，需要所有项都 pass）
 - 有 vdo 数据的 Dphy 波形



- 没有 vdo 数据的 Dphy 波形



- Cphy 波形



- 是否有放 logo 文件？若无 logo 文件，屏上只会显示一行小字，需要仔细观察

1.8.2 Kernel 阶段 Debug

- 进 Kernel 后屏不亮
 - 量屏的各路电压是否有掉

- 进 Kernel 后 regulator 在没人使能会自动关掉，屏驱中需要获取资源并使能它
- 背光是否有亮？
 - AAL 可能会导致背光关闭，关闭 AAL 看看是否有问题
 - 调整背光的函数是否能跑到，如不能需要加 log debug
- Kernel 屏驱是否有正常加载？
 - 观察 log，看驱动加载是否正常。Kernel 运行过程中需要一些参数
- 是否有使能 esd，esd 也会导致黑屏
- 是否有多种 mode 切换？Bring up 阶段先点一个 mode

1.8.3 Resume 阶段 Debug

- Resume 后屏不亮
 - 量屏的各路电压是否有正常起来
 - 检查屏的 power on code 是否有异常
 - 背光是否有亮？
 - 控制背光 code 是否有异常
 - Kernel 屏驱是否有正常加载？
 - 观察 log，看驱动加载是否正常。
 - 对比跟 LK 的驱动，看两边设置是否一样

1.8.4 花屏 Debug

- 是否有开 dsc
 - 是，检测 DSC 设置
 - 打 DSC test pattern 是否能显示正常（echo reg_write 0x14015078 0x400000FF > /proc/clkgdbg ; cat /proc/clkgdbg）
 - 0x14015xxx 是 dsc base reg，根据 soc 不同而不同，请询问 MTK 工程师正确的值
 - 0x078 是 dsc test pattern reg 地址，根据 soc 不同而不同，请询问 MTK 工程师正确的值
 - 能，找 display owner 寻求帮助
 - 不能，是否存在正常显示时刻，有正常时刻。dump dsc reg 对比是否有设置不一样
 - Dump 命令：adb shell "echo mobile:on > /d/mtkfb";adb shell "echo diagnose>/d/mtkfb && cat /d/mtkfb" > mtkfb.txt 搜索 DSC 部分

- 典型 dsc 花屏图片：



- 类似花屏是 AFBC 导致，使用如下命令关闭 AFBC 看看：

```
adb shell setprop debug.mediatek.disp_decompress 0
adb shell stop
adb shell start
```

- 典型的 AFBC 花屏：



- 打 DSI test pattern 是否能正常显示

- 命令（参考 dump reg）：echo reg_write 0x14017178 0x31 > /proc/clktb; cat /proc/clktb

- 0x1401xxxx 是 dsi reg 地址，根据 soc 不同而不同，请咨询 MTK 工程师
- 0x178 是 dsi test pattern 地址，根据 soc 不同而不同，请咨询 MTK 工程师
- 能正常显示，找 display owner 寻求帮助
- 不能，log 中是否存在 dsi underrun 字样

- 是否有显示正常的情况

- 是，dump 正常和不正常时的 DSI/MIPITX/DSC reg 值，看设置是否有不一样
- Dump 命令：adb shell "echo mobile:on > /d/mtkfb";adb shell "echo diagnose>/d/mtkfb && cat /d/mtkfb" > mtkfb.txt 搜索 DSI 部分
- Dump 命令：参考 dump reg 这一块，用 adb cmd 的方法读取也可以

- 是否经过某种操作后才出现（比如切换 fps）

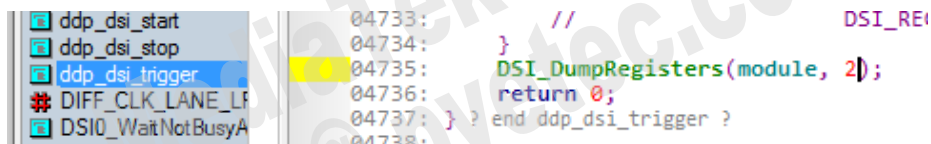
- 对比切换前后 reg 是否一样
- 例：切换 fps 后花屏
 - 询问屏厂切换 fps 是否存在限制，是否要发送 cmd 等
 - Fps 切换时 mmclk 改变先后顺序是否正确（mmclk 变高要先设 mmclk 后切 fps，mmclk 变低要先切 fps 后设置 mmclk，mmclk 计算方法参考 mtk_dsi_set_mmclk_by_datarate 函数，切换逻辑参考 mtk_crtc_disp_mode_switch_begin 函数）

1.8.5 含有 Bridge 屏的 Debug

- 查看 bridge 状态寄存器，可以得知当前 bridge 工作状态
- 从 bridge vendor 获取帮助

1.9 Dump Reg

1.9.1 LK 阶段

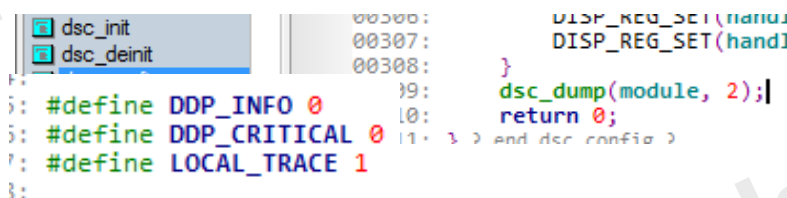


```

04733: // DSI_REG
04734:
04735: DSI_DumpRegisters(module, 2);
04736: return 0;
04737: } ? end ddp_dsi_trigger ?
04738:

```

- 在 Ddp_dsi.c/ddp_dsi_trigger() 最后调用 DSI_DumpRegister()
- 在 Ddp_dsi.c/dsc_config() 最后调用 dsc_dump()
- LK display log 默认没开，需要在 ddp_log.h 中打开



```

00306: DISP_REG_SET(handl
00307: DISP_REG_SET(handl
00308:
00309: }
00310: dsc_dump(module, 2);|
00311: return 0;
00312: } ? end dsc_config ?
00313:

```

```

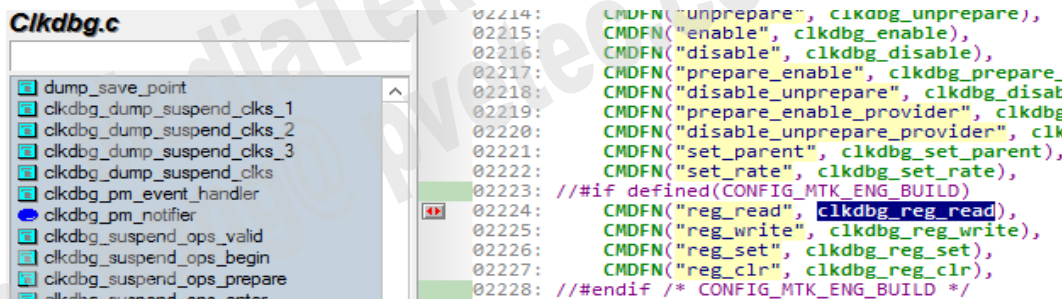
#define DDP_INFO 0
#define DDP_CRITICAL 0
#define LOCAL_TRACE 1

```

1.9.2 Kernel Dump reg

- 如何打开寄存器 adb debug 接口
 - 路径：kernel-xx/drivers/clk/mediatek/clktb.c/common_cmds[]
 - 函数：clktb_reg_read()和 clktb_reg_write()

- 方法：如果 `clkdbg_reg_read()` and `clkdbg_reg_write()` 被 `#if defined( CONFIG_MTK_ENG_BUILD)` 包起来了，请把 `#if defined( CONFIG_MTK_ENG_BUILD)` mask 掉，同时把定义 `clkdbg_reg_read()` and `clkdbg_reg_write()` 地方的 `#if defined(CONFIG_MTK_ENG_BUILD)` 也 mask 掉，然后就可以用 adb command 改寄存器了



1.9.3 adb 命令 dump reg

- 寄存器读写命令

- 读: `echo reg_read addr> /proc/clkdbg ; cat /proc/clkdbg`
 - 按长度读: `echo reg_read_len addr length> /proc/clkdbg; cat /proc/clkdbg`
- 写: `echo reg_write addr value> /proc/clkdbg; cat /proc/clkdbg`
 - 举例:


```
echo reg_read 0x1400D178 > /proc/clkdbg ; cat /proc/clkdbg
echo reg_write 0x1400D178 0x31 > /proc/clkdbg ; cat /proc/clkdbg
echo reg_read_len 0x1400D000 0x00001000 > /proc/clkdbg ; cat /proc/clkdbg
```
 - 注意:
 - 通常 MTK supporter 会说请把 xxx 的 yyy 写 zzz，其中 xxx 代表某个模块，通过前面 Debug SOP 章节获取该模块的基地址，yyy 代表偏移量，zzz 代表 value
 - 该语句最终需要执行的命令是: `echo reg_write xxx 基地址+yyy zzz> /proc/clkdbg; cat /proc/clkdbg`
 - 例如调整 mipi 的驱动能力，supporter 会说把 mipi_tx 的 0x10 bit6-9 写 0111
 - 执行的命令是: `echo reg_read 0x11f60010 > /proc/clkdbg; cat /proc/clkdbg, //mipi_tx 基地址 0x11f60000，先读 mipi tx 的 0x10，假如读到是 0，将 0 的 bit6-9 或上 0111 得到 0x1c0: echo reg_write 0x11f60010 0x1c0> /proc/clkdbg; cat /proc/clkdbg`

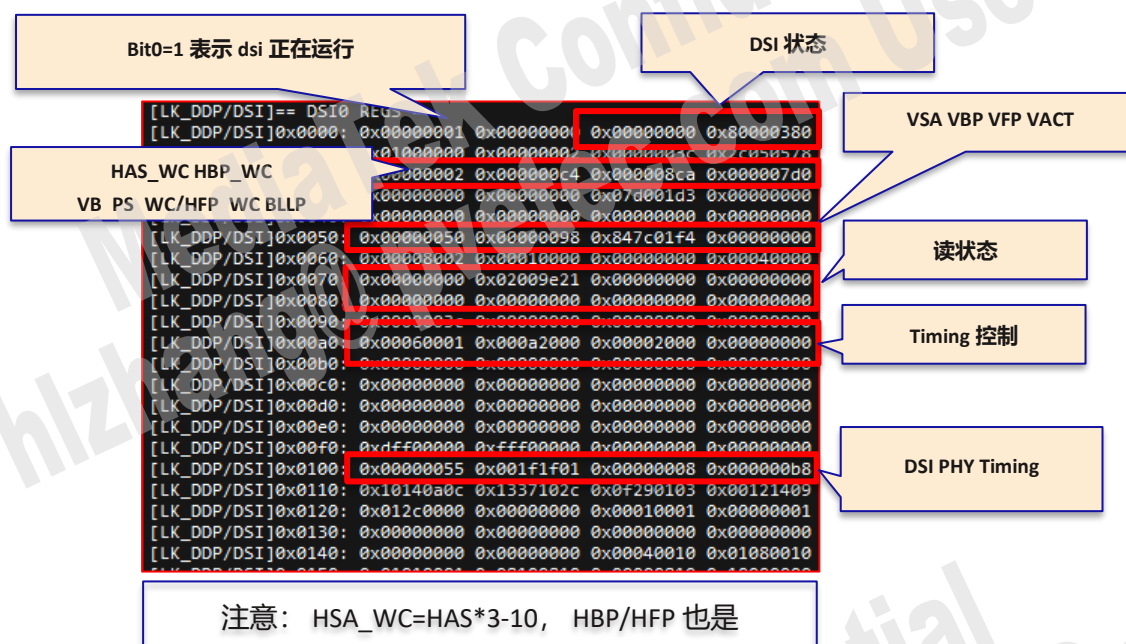
1.9.4 Test Pattern 命令

- 使用 adb:
 - `echo reg_write "DSI/DSC_base_reg+offset_addr" value> /proc/clkdbg; cat /proc/clkdbg`
 - DSI/DSC 基地址参考前面的说明

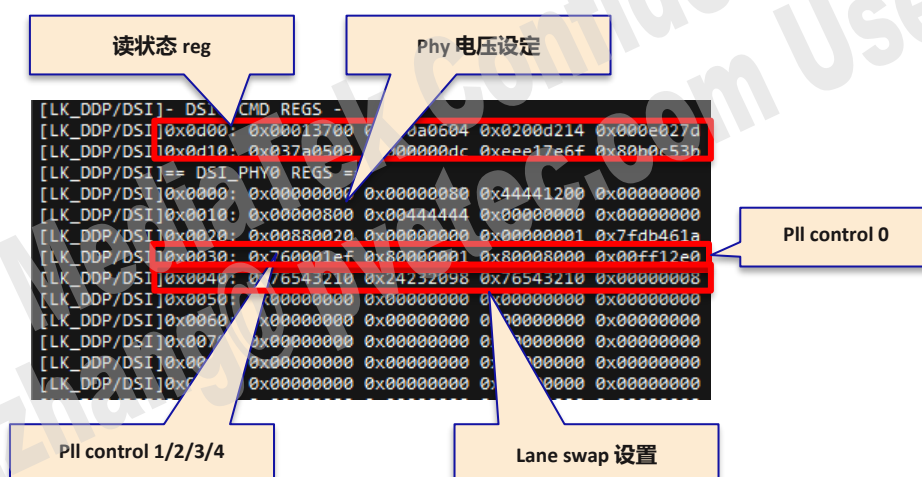
- Offset:
 - Dsi:0x178 or 0x17c
 - Dsc:0x78 or 0x6c, 询问 MTK 工程师确认
- Value:
 - Dsi: 0xc41
 - Dsc: 0x4000FFFF(blue), 询问 MTK 工程师确认
- 在 code 中加入:
 - DSI: 在 Ddp_dsi_trigger 函数最后调用 DSI_OUTREG32(NULL, DSI_REG_BASE[0]+offset, value);
 - DSC: 在 dsc_config 函数最后调用 DISP_REG_SET(handle, base+offset, value);
 - Offset/value 参考 adb 命令

1.9.5 分析 reg

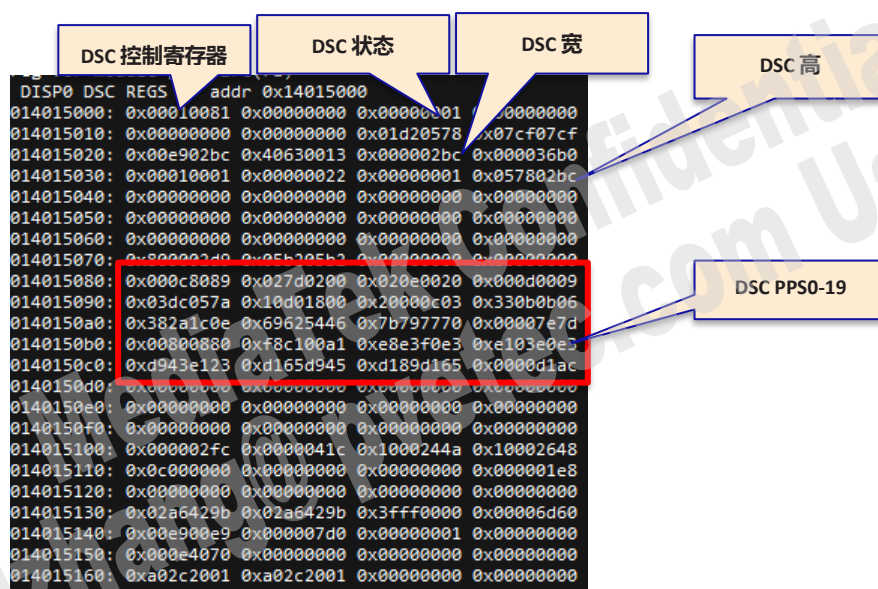
- 如下 reg 说明, 仅供参考, 具体 reg 内容请参考 coda
- DSI



- MIPI TX



- DSC



1.10 使用 MTK max96789 panel driver

MTK 提供了一个 max96789+max96752 的通用驱动，可参考这个通用驱动写自己的驱动：

1. KO table 中加入 birdge-serdes-max96789.ko/panel-serdes-max96789.ko (如果不存在时)
2. LK 的 project.mk 中设置 MTK_LCM_LIST_SUPPORT="max96789_dsi_vdo"
3. kleaf 中加入 birdge-serdes-max96789.ko/panel-serdes-max96789.ko (如果不存在时)
4. defconfig 中加入 CONFIG_DRM_PANEL_SERDES=m (如果不存在时)
5. 如果需要使用 hotplug 功能，需要在 defconfig 中设置 CONFIG_ENABLE_SERDES_HOTPLUG=y
6. 如果 hotplug 功能要使用中断方式，需要在 code 中将 ENABLE_HOTPLUG_INT 定义为 1 (同时要修改 dws 将对于 gpio 设置为 EINT 功能)

7. dts 中加入如下设定

8. 其他设定均在 driver 中 hard code 写死, 如有不满足需求可以剥离出来写到 dts

```
&i2c0 {                                     // max96789 在哪个 IIC 下面, 必须填写
    status = "okay";
    max96789: max96789@40 {                 // max96789 的 node, 必须填写
        compatible = "maxim,max96789";
        reg = <0x40>;                       // max96789 的 IIC 地址, 必须填写
        reset-gpios = <&pio 24 0>;          // max96789 的 pwn 脚, 必须填写
        interrupt-parent = <&pio>;
        interrupts = <187 IRQ_TYPE_EDGE_RISING>; // 使用中断方式检测 hotplug 时必须填写, 否则可以不写
        pinctrl-names = "default";
        initd-in-lk = <0>;                 // 这个设定是否会在 LK 设定, 如果会, 设置为 1, Kernel 启动时不会再初始化它
        setting = <&setting_compatible>;    // 这个串行器接的是哪个屏就写该屏的名称, 如果需要多屏兼容写 setting_compatible (注意现在多屏兼容是通过读 mcu 来实现, 如果不是这种方式需要自己实作), 必须填写
    };
};

&max96789 {
    setting_compatible: compatible-node {    // 使用多屏兼容时的 node, 使用多屏兼容时必须填写
        comp-cmd = <
            0x01 0x1a 0x0b 0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00
            0xd4 >;                          // 多屏兼容时写的命令, 格式是: <r/w><iic addr><cmd len><cmd data>, 注意, 这种方式只是适合公版使用的博泰/华阳屏, 客户应该根据实际情况定制 (包括修改代码)
            // r/w: 读=0, 写=1
            // iic addr: 要读的 iic 设备的地址
            // cmd len: 这一条命令的数据长度 (可以是多条写命令)
            // cmd data: 写的命令数据
            0x00 0x1a 0x0b >;               // 读命令, 格式是: <r/w><iic addr><cmd len>, 含义同上。注意读命令必须有, 且放在最后一个, 并且只能读一次
        };
        comp-exp {                          // 预期的设定 node, 使用多屏兼容时必须填写
            comp-setting = <0xf0 0x10 &setting_bt>; // 预期的多屏兼容设置, 格式为: <mask><exception data><panel setting node>, 使用多屏兼容时必须填写
        };
    };
};
```

```

// mask: 读出来的值跟 mask 做与操作, 再跟
exception data 做对比, 如果一致, 则使用该行 panel setting
<0xff 0xff &setting_hy>; // 注意最后一行是默认设置, 即如果读
出的数据都是非预期的, 将会使用这个设定
};
};
setting_bt: setting@1 { // panel setting, 至少需要设置 1 个
    ser-super-frame = <0>; // 是否为 super frame 设置, super frame
会有两个 des, 计算 timing 也不一样
    ser-init-cmd = <
<reg><data><delay>
    0x0001 0x08 0x00
    0x0203 0x00 0x00 // reg: 要写的 reg
    0x1404 0x29 0x00 // data: 要写的数据
    0x1504 0x29 0x00 // delay: 写完该命令后 delay 多久, 单位 ms
    0x14a4 0xc8 0x00
    0x15a4 0xc8 0x00
    0x140a 0x00 0x00
    0x150a 0x00 0x00
    0x140b 0x00 0x00
    0x150b 0x00 0x00
    0x0330 0x04 0x00
    0x0331 0x33 0x00
    0x0333 0xe4 0x00
    0x0308 0x7c 0x00
    0x0053 0x10 0x00
    0x005b 0x12 0x14
    0x0140 0x20 0x00
    0x0002 0x53 0x05
    0x0010 0x31 0xc8
>;
    ser-deinit-cmd = < // 串行器 deinit 命令, 复位所有 reg
    0x0010 0x80 0x20
>;
    des-link-status-cmd = < // 支持 hotplug 功能时必须填写, 用来检测解
串器是否有插上
    0x001f 0x18 0x0
>;
    desdef { // 默认解串器 node, 必须要写
        des-i2c-addr = <0x4c>; // 默认解串器 i2c 地址, 必须要写
        bl-i2c-addr = <0x1a>; // 默认解串器对应的背光 i2c 地址, 按需填写
        bl-dummy-i2c-addr = <0x1f>; // 带 mcu 的屏, 背光跟 tp 都是由 mcu 控制,
导致他俩的 iic 地址一样, 这里将 mcu 实际 iic 地址让给 tp 用, 背光用这个地址在 kernel 中获取
client, 使用 client 时再将地址设置为实际地址。仅仅在 kernel 中用到

```

```

des-init-cmd = <                                // 解串器初始化命令，按需填写，格式同串行
器初始化命令
0x01ce 0x5e 2
0x06ff 0x22 0x0                                // 支持 hotplug 功能时必须加上这行，通过读
取此 reg 设定来判断解串器是否有被初始化过，注意必须跟 link-indicate-cmd 一致
>;
link-indicate-cmd = <                            // 支持 hotplug 功能时必须填写，通过读取此
设定，用来判断解串器是否有被初始化过，注意必须跟 des-init-cmd 中的某项一致
0x06ff 0x22 0x0
>;
bl-on-cmd = <                                    // 背光开命令，格式为全部 data，仅适合公版
博泰屏，按需填写
0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0xd4
>;
bl-off-cmd = <                                    // 背光关命令，格式为全部 daa，按需填写
0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00 0x85
>;
panel-mode-setting {                               // 屏的 timing 信息，必须填写
    panel-mode-width = <1920>;                    // 宽
    panel-mode-height = <1080>;                   // 高
    panel-mode-hfp = <40>;                         // HFP
    panel-mode-hsa = <40>;                         // HSA
    panel-mode-hbp = <80>;                         // HBP
    panel-mode-vfp = <24>;                         // VFP
    panel-mode-vsa = <2>;                         // VSA
    panel-mode-vbp = <10>;                        // VBP
    panel-mode-vrefresh = <60>;                   // FPS 刷新率
    panel-mode-pll = <478>;                        // PLL, mipi 的 clock, 系统会按 timing
计算一个 pll，如果发现有偏差，可以通过设置它来替换系统计算结果。可选
    panel-mode-lppf = <1>;                        // enter LP per frame, 每帧进 LP，默认
为 1，可选
    panel-mode-width-mm = <129>;                  // 屏的物理尺寸，用来计算 PPI
    panel-mode-height-mm = <64>;
};
};
setting_hy: setting@2 {
    ser-dual-link = <0>;

    ser-init-cmd = <
        0x0001 0x08 0x00
        0x0203 0x00 0x00
        0x1404 0x29 0x00
        0x1504 0x29 0x00
        0x14a4 0xc8 0x00

```

```

0x15a4 0xc8 0x00
0x140a 0x00 0x00
0x150a 0x00 0x00
0x140b 0x00 0x00
0x150b 0x00 0x00
0x0330 0x04 0x00
0x0331 0x33 0x00
0x0333 0xe4 0x00
0x0308 0x7c 0x00
0x0053 0x10 0x00
0x005b 0x12 0x14
0x0140 0x20 0x00
0x0002 0x53 0x05
0x0010 0x31 0xc8
>;
desdef {
    des-i2c-addr = <0x4c>;
    bl-i2c-addr = <0x1a>;
    des-init-cmd = <
        0x01ce 0x4e 2
    >;
    bl-on-cmd = <
        0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0xd4
    >;
    bl-off-cmd = <
        0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00 0x85
    >;
    panel-mode-setting {
        panel-mode-width = <1920>;
        panel-mode-height = <1080>;
        panel-mode-hfp = <40>;
        panel-mode-hsa = <40>;
        panel-mode-hbp = <80>;
        panel-mode-vfp = <24>;
        panel-mode-vsa = <2>;
        panel-mode-vbp = <10>;
        panel-mode-vrefresh = <60>;
        panel-mode-pll = <478>;
        panel-mode-lppf = <1>;
        panel-mode-width-mm = <129>;
        panel-mode-height-mm = <64>;
    };
};

superframe_setting: setting@3 {
    ser-super-frame = <1>;
    ser-init-cmd = <
        0x0002 0x73 0x0
        0x0053 0x10 0x0
        0x0057 0x21 0x0
        0x0332 0x4E 0x0
    >;
    // super frame setting demo
    // 需要设置 super frame 为 1
    // 同上含义

```

```

0x0333 0xE4 0x0
0x0004 0xF2 0x0
0x0308 0x5C 0x0
0x0311 0x03 0x0
0x0331 0x03 0x0
0x0330 0x06 0x0
0x031C 0x98 0x0
0x0321 0x24 0x0
0x031D 0x98 0x0
0x0322 0x24 0x0
0x0326 0xE4 0x0
0x03A4 0xC1 0x0
0x032A 0x07 0x0
0x0002 0x73 0x5
>;

ser-timing-cmd = <                                     // super frame 必须设置 timing 参
数, 必填
0x385 0x50 0x0
0x386 0x02 0x0
0x387 0x00 0x0
0x3A5 0x18 0x0                                           // 注意这里的 reg 顺序不能改变, 现在的
顺序是按照串行器 tool gen 出来的顺序
0x3A7 0x00 0x0
0x3A6 0xA0 0x0
0x3A8 0x38 0x0
0x3A9 0x04 0x0
0x3AA 0x50 0x0
0x3AC 0x0A 0x0
0x3AB 0x00 0x0
0x3AD 0x00 0x0
0x3AE 0x0f 0x0
>;

serdes-dual-setting-cmd = <                               // 同样的屏拥有同样的 des iic
addr/bl iic addr 等, 通过此命令将其区分开
0 0x0010 0x21 255                                       // 选择 linka
1 0x0000 0x90 0                                           // 0x90=desa-i2c-addr*2, 设置
desa 的 des i2c addr 为 0x90 (8bit 地址)
2 0x0073 0x31 0                                           // 设置为 split mode, 且当前
channel id 为 1
2 0x0042 0x36 0                                           // 0x36=desa-bl-i2c-addr*2
2 0x0043 0x34 0                                           // 0x34=desdef-bl-i2c-addr*2, 这
段含义是将 linka 上面的 bl iic 地址由 0x34 改为 0x36, 注意 0x36 一定要跟 desa 里面的 des-i2c-
addr 保持一致
0 0x0010 0x22 255                                       // 选择 linkb
1 0x0000 0x94 0                                           // 0x94=desb-i2c-addr*2, 设置
desb 的 des i2c addr 为 0x94 (8bit 地址)

```

```

3 0x0073 0x32 0 // 设置为 split mode, 且当前
channel id 为 2
3 0x0042 0x38 0 // 0x38=desb-bl-i2c-addr*2
3 0x0043 0x34 0 // 0x34=desdef-bl-i2c-addr*2
0 0x0010 0x23 255 // 选择所有通道
3 0x0050 0x01 0 // 设置 linkb 的 stream id 为 1
>;
ser-lut-cmd = < // 非对称 super frame 会有 lut 设
置, 写在这里
>;
ser-deinit-cmd = <
0x0010 0x80 0x20
>;
des-link-status-cmd = <
0x001f 0x18 0x0
>;
desdef { // 默认 des 配置, 必填
    des-i2c-addr = <0x4c>; // des 默认 i2c addr
    bl-i2c-addr = <0x1a>; // des 默认背光 i2c addr, 注意要跟
dual-setting-cmd 中的 0x0043 后面的 data 匹配(8bit i2c addr)
};
desa { // linka 的 des 配置, super frame
    des-i2c-addr = <0x48>; // linka des i2c addr, 注意跟
dual-setting-cmd 中的 0x0000 的 data 匹配
    bl-i2c-addr = <0x1b>; // linka bl i2c addr, 注意跟
dual-setting-cmd 中的 0x0042 的 data 匹配
    des-init-cmd = < // linka des 初始化命令
0x01ce 0x5e 2
0x06ff 0x22 0x0 // 支持 hotplug 功能时必须加上这行,
通过读取此 reg 设定来判断解串器是否有被初始化过, 注意必须跟 link-indicate-cmd 一致
>;
link-indicate-cmd = < // 支持 hotplug 功能时必须填写, 通过
读取此设定, 用来判断解串器是否有被初始化过, 注意必须跟 des-init-cmd 中的某项一致
0x06ff 0x22 0x0
>;
bl-on-cmd = < // 同 single link
0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0xd4
>;
bl-off-cmd = < // 同 single link
0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00 0x85
>;

```

```

panel-mode-setting {                                     // 同 single link
    panel-mode-width = <1920>;
    panel-mode-height = <1080>;
    panel-mode-hfp = <40>;                               // 注意必须保证 (hfp-12)/4 是整数
    panel-mode-hsa = <41>;                               // 注意必须保证
    (desa_hsa+desb_hsa-10)/4 是整数
    panel-mode-hbp = <79>;                               // 注意必须保证
    (desa_hbp+desb_hbp-10)/4 是整数
    panel-mode-vfp = <24>;                               // 注意非对称 superframe 必须保证
    desa_vfp/desb_vfp=desa_height/desb_height
    panel-mode-vsa = <3>;                               // 注意非对称 superframe 必须保证
    desa_vsa/desb_vsa=desa_height/desb_height
    panel-mode-vbp = <9>;                               // 注意非对称 superframe 必须保证
    desa_vbp/desb_vbp=desa_height/desb_height
    panel-mode-vrefresh = <60>;
    panel-mode-pll = <478>;                             // superframe 不会拿这个设定
    panel-mode-lppf = <1>;                               // super frame 不会拿这个设定, 必
    须为 1
    panel-mode-width-mm = <129>;
    panel-mode-height-mm = <64>;
};
desb {                                                  // 同 linka
    des-i2c-addr = <0x4a>;
    bl-i2c-addr = <0x1c>;
    des-init-cmd = <
        0x01ce 0x5e 2
        0x06ff 0x22 0x0
    >;
    link-indicate-cmd = <
        0x06ff 0x22 0x0
    >;
    bl-on-cmd = <
        0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0xd4
    >;
    bl-off-cmd = <
        0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00 0x85
    >;
    panel-mode-setting {
        panel-mode-width = <1920>;
        panel-mode-height = <1080>;
        panel-mode-hfp = <40>;
        panel-mode-hsa = <40>;
        panel-mode-hbp = <80>;
        panel-mode-vfp = <24>;
        panel-mode-vsa = <2>;
        panel-mode-vbp = <10>;
        panel-mode-vrefresh = <60>;
    }
};

```

```
        panel-mode-pll = <478>;
        panel-mode-lppf = <1>;
        panel-mode-width-mm = <129>;
        panel-mode-height-mm = <64>;
    };
};

};

};
&dsi0 {
    status = "okay";
    #address-cells = <1>;
    #size-cells = <0>;
    panel1@0 {
        compatible = "lcm,dsi,max96789";           // panel 设定
        reg = <0>;
        ser = <&max96789>;                          // panel 对应哪个串行器, 必填
        port {
            panel_in1: endpoint {
                remote-endpoint = <&dsi_out>;
            };
        };
    };
};
};
```

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿赔偿责任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。